ZeroPad1d#

https://pytorch.org/docs/stable/generated/torch.nn.ZeroPad1d.html

Description:

Pads the input tensor boundaries with zero. For N-dimensional padding, use torch.nn.functional.pad(). padding (int, tuple) – the size of the padding. If is int, uses the same padding in both boundaries. If a 2-tuple, uses (padding_left\text{padding_left}padding_left, padding_right\text{padding_right}padding_right) Input: (C,Win)(C, W_{in})(C,Win) or (N,C,Win)(N, C, W_{in})(N,C,Win). Output: (C,Wout)(C, W_{out})(C,Wout) or (N,C,Wout)(N, C, W_{out})(N,C,Wout), where

Function Signature

```
class torch.nn.ZeroPad1d(padding)[source]#

Parameters

Shape:

extra_repr()[source]#

Return type
```

Returns:

str

Code Examples

```
>>> m = nn.ZeroPad1d(2)
>>> input = torch.randn(1, 2, 4)
>>> input
tensor([[[-1.0491, -0.7152, -0.0749,  0.8530],
         [-1.3287,  1.8966,  0.1466, -0.2771]]])
>>> m(input)
tensor([[[ 0.0000,  0.0000, -1.0491, -0.7152, -0.0749,  0.8530,
0.0000,
           0.0000],
         [ 0.0000,  0.0000, -1.3287,  1.8966,  0.1466, -0.2771,
0.0000,
           0.0000]]])
>>> m = nn.ZeroPad1d(2)
>>> input = torch.randn(1, 2, 3)
>>> input
tensor([[[ 1.6616,  1.4523, -1.1255],
         [-3.6372,  0.1182, -1.8652]]])
>>> m(input)
tensor([[[ 0.0000,  0.0000,  1.6616,  1.4523, -1.1255,  0.0000,
0.0000],
         [ 0.0000,  0.0000, -3.6372,  0.1182, -1.8652,  0.0000,
0.0000]]])
>>> # using different paddings for different sides
>>> m = nn.ZeroPad1d((3, 1))
>>> m(input)
tensor([[[ 0.0000,  0.0000,  0.0000,  1.6616,  1.4523, -1.1255,
0.0000],
         [ 0.0000,  0.0000,  0.0000, -3.6372,  0.1182, -1.8652,
0.0000]]])
```

Detailed Documentation

Documentation

Pads the input tensor boundaries with zero.

For N-dimensional padding, use torch.nn.functional.pad().

padding (int, tuple) – the size of the padding. If is int, uses the same padding in both boundaries. If a 2-tuple, uses (padding_left\text{padding_left}padding_left, padding_right\text{padding_right}padding_right)

Input: (C,Win)(C, W_{in})(C,Win) or (N,C,Win)(N, C, W_{in})(N,C,Win).

Output: (C,Wout)(C, W_{out})(C,Wout) or (N,C,Wout)(N, C, W_{out})(N,C,Wout), where

Wout=Win+padding_left+padding_rightW_{out} = W_{in} + \text{padding_left} + \text{padding_right}Wout=Win+padding_left+padding_right

Return the extra representation of the module.